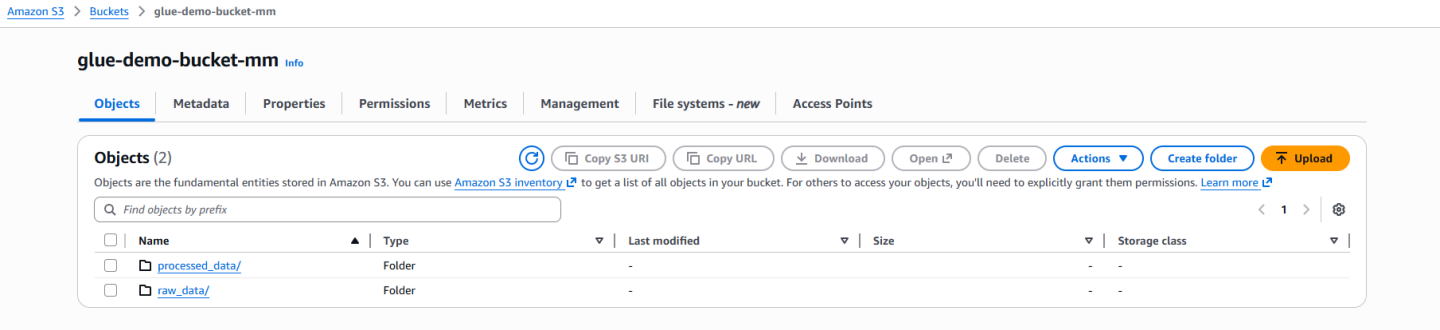
Name :- Mihir Mungara

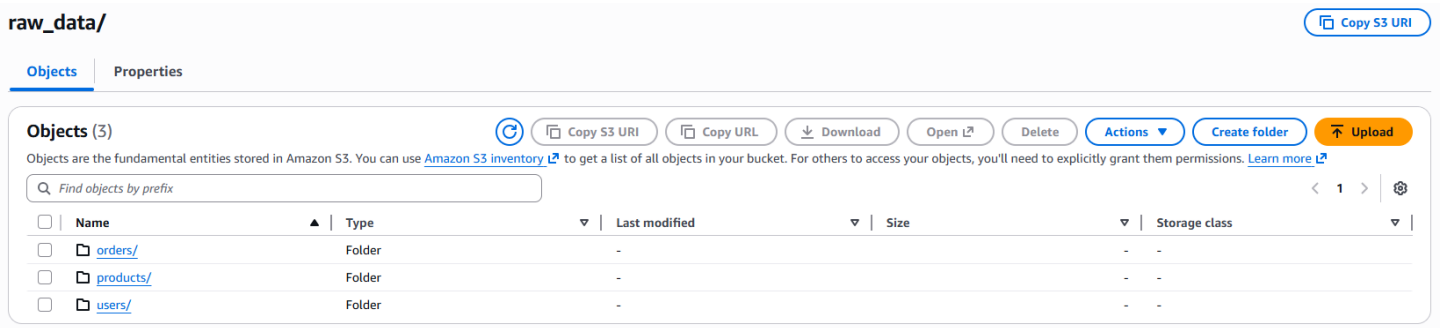
AWS Glue Assignment

The objective of this assignment is to:

- Ingest raw data into Amazon S3
- Automatically catalog data using AWS Glue Crawlers
- Perform ETL (Extract, Transform, Load) using AWS Glue
- Store processed data in Parquet format with partitioning
- Query processed data using Amazon Athena
- Load transformed data into Amazon Redshift for analytics

S3 Bucket :-





The S3 bucket is divided into two main layers:

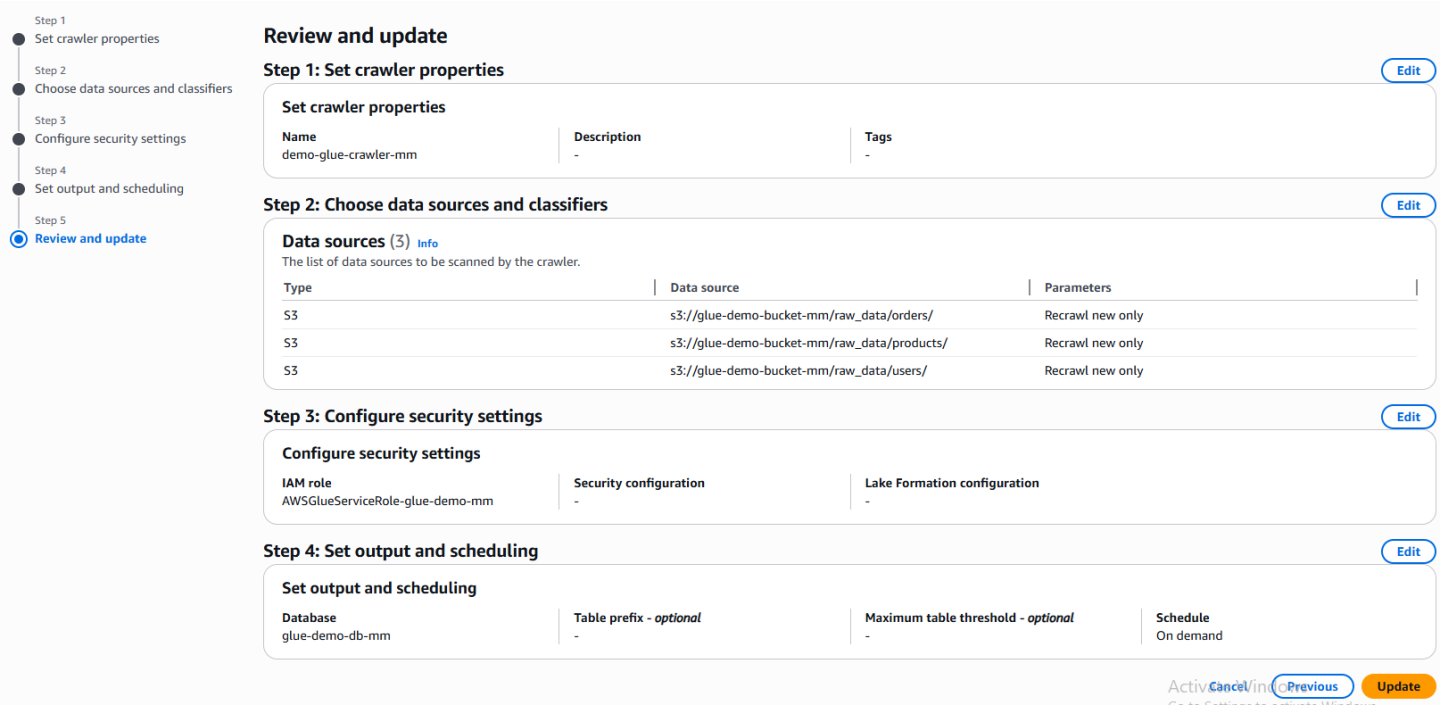
1. Raw Data Layer (raw_data/)

- Contains unprocessed CSV files
- Organized into:
  - users/
  - orders/
  - products/

2. Processed Data Layer (processed_data/)

- Stores transformed data in Parquet format
- Partitioned by OrderDate for efficient querying

Crawler :-



The crawler is configured to scan the raw_data folder in S3 and automatically infer schema.

Key functions:

- Detects file format (CSV)

- Infers column names and data types

- Creates tables in Glue Data Catalog

Output:

- Database: glue-demo-db-mm

- Tables created:

  - users

  - orders

  - products

This eliminates the need for manual schema definition.

ETL Process using AWS Glue:

The ETL pipeline performs the following steps:

1. Data Loading:

- Data is loaded from Glue Catalog into Spark DataFrames
- This enables distributed processing using Apache Spark

2. Data Cleaning:

- Removed records with null values in critical fields such as:
  - userid
  - productid
  - quantity
- Ensures data quality and consistency

3. Date Formatting:

- Handles multiple date formats (yyyy-MM-dd and yyyy/MM/dd)
- Converts them into standard DATE format

4. Data Validation:

- Records with invalid or null dates are removed

5. Data Joining:

- Orders joined with Users on userid
- Result joined with Products on productid
- Creates a unified dataset

6. Feature Engineering:

- CustomerAge = current_date - DateOfBirth

- TotalPrice = quantity × price

7. Filtering:

- Only high-value orders retained:
  - quantity > 3
  - TotalPrice > 50

8. Final Output:

- Selected only relevant columns for analytics

ETL Script :-

Script Info

```python
import sys
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from pyspark.sql.functions import col, to_date, coalesce, datediff, current_date, floor

# Initialize
sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session

# --------------------------
# LOAD DATA FROM CATALOG
# --------------------------

users_df = glueContext.create_dynamic_frame.from_catalog(
    database='glue-demo-db-mm',
    table_name='users'
).toDF()

orders_df = glueContext.create_dynamic_frame.from_catalog(
    database='glue-demo-db-mm',
    table_name='orders'
).toDF()

products_df = glueContext.create_dynamic_frame.from_catalog(
    database='glue-demo-db-mm',
    table_name='products'
).toDF()
```

Script Info

```python
# --------------------------
# INITIAL CLEANING
# --------------------------

users_df = users_df.dropna(subset=['userid', 'dateofbirth'])
orders_df = orders_df.dropna(subset=['userid', 'productid', 'quantity', 'OrderDate'])
products_df = products_df.dropna(subset=['productid', 'price'])

# --------------------------
# FORMAT DATES
# --------------------------

users_df = users_df.withColumn(
    "DateOfBirth",
    coalesce(
        to_date(col("dateofbirth"), "yyyy-MM-dd"),
        to_date(col("dateofbirth"), "yyyy/MM/dd")
    )
)

orders_df = orders_df.withColumn(
    "OrderDate",
    coalesce(
        to_date(col("OrderDate"), "yyyy-MM-dd"),
        to_date(col("OrderDate"), "yyyy/MM/dd")
    )
)
```

Script Info

```python
# --------------------------
# REMOVE INVALID DATES
# --------------------------

users_df = users_df.dropna(subset=["DateOfBirth"])
orders_df = orders_df.dropna(subset=["OrderDate"])

# --------------------------
# JOIN DATA
# --------------------------

joined_df = orders_df \
    .join(users_df, "userid") \
    .join(products_df, "productid")

# --------------------------
# ADD NEW FIELDS
# --------------------------

joined_df = joined_df.withColumn(
    "CustomerAge",
    floor(datediff(current_date(), col("DateOfBirth")) / 365)
)

joined_df = joined_df.withColumn(
    "TotalPrice",
    col("quantity") * col("price")
)
```

Script Info

```python
# --------------------------
# FILTER DATA
# --------------------------

final_df = joined_df.filter(
    (col("quantity") > 3) & (col("TotalPrice") > 50)
)

# --------------------------
# SELECT OUTPUT
# --------------------------

final_df = final_df.select(
    "orderid",
    "userid",
    "firstname",
    "lastname",
    "productname",
    "quantity",
    "price",
    "TotalPrice",
    "CustomerAge",
    "OrderDate"
)
```

```python
# --------------------------
# WRITE TO S3
# --------------------------

output_path = "s3://glue-demo-bucket-mm/processed_data/"

final_df.write \
    .mode("overwrite") \
    .format("parquet") \
    .partitionBy("OrderDate") \
    .save(output_path)
```

```python
# -------------------------
# Redshift Part
# -------------------------
from awsglue.dynamicframe import DynamicFrame

# Convert to DynamicFrame
final_dyf = DynamicFrame.fromDF(final_df, glueContext, "final_dyf")

# Write to Redshift
glueContext.write_dynamic_frame.from_options(
    frame=final_dyf,
    connection_type="redshift",
    connection_options={
        "dbtable": "public.customer_orders",
        "database": "dev",
        "connectionName": "Jdbc connection",
        "redshiftTmpDir": "s3://glue-demo-bucket-mm/temp/",
        "useConnectionProperties": "true"
    }
)
```

Querying Using Athena :-

Query 1 | Query 2

```sql
CREATE EXTERNAL TABLE processed_data (
    orderid INT,
    userid INT,
    firstname STRING,
    lastname STRING,
    productname STRING,
    quantity INT,
    price INT,
    totalprice INT,
    customerage INT,
    orderdate DATE
)
STORED AS PARQUET
LOCATION 's3://glue-demo-bucket-mm/processed_data/';
```

Query 1 | Query 2

```sql
SELECT * FROM processed_data;
```

Results (5)

Copy | Download results CSV

Search rows

#	orderid	userid	firstname	lastname	productname	quantity	price	totalprice	customerage	orderdate
1	116	1	John	Doe	Phone	5	600	3000	30	
2	114	7	Tom	Holland	Shoes	4	50	200	29	
3	109	8	Robert	Downey	Watch	4	120	480	56	
4	113	6	Chris	Evans	Watch	5	120	600	37	
5	106	5	Emma	Watson	Hat	5	15	75	32	

Athena is used to query processed data directly from S3 using SQL

Steps:

- Created external table on top of S3 Parquet data

- Schema defined to match transformed dataset

- Queries executed without moving data

Processed Data in S3 :-

processed_data/

Copy S3 URI

Objects | Properties

Objects (5)

Copy S3 URI | Copy URL | Download | Open | Delete | Actions | Create folder | Upload

Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more

Find objects by prefix

Name	Type	Last modified	Size	Storage class
OrderDate=2024-05-06/	Folder	-	-	-
OrderDate=2024-05-09/	Folder	-	-	-
OrderDate=2024-05-12/	Folder	-	-	-
OrderDate=2024-05-13/	Folder	-	-	-
OrderDate=2024-05-15/	Folder	-	-	-

OrderDate=2024-05-06/

Copy S3 URI

Objects | Properties

Objects

Copy S3 URI | Copy URL | Download | Open | Delete | Actions | Create folder | Upload

Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more

Find objects by prefix

Name	Type	Last modified	Size	Storage class
part-00000-b1a7cd7e-e54f-435f-a414-9a21ab45a1b4.c000.snappy.parquet	parquet	April 17, 2026, 14:05:12 (UTC+05:30)	2.5 KB	Standard

Redshift part :-

Jdbc connection

Edit | Delete | Create job

Connection details Info

Connector type
JDBC

Connection URL
jdbc:redshift://demo-glue-redshift-cluster.cefcvyldynkj.us-east-1.redshift.amazonaws.com:5439/dev

Driver class name
-

Driver path
-

Username
-

Require SSL connection
-

Subnet
subnet-0d23367a87fb3ab12

Security groups
sg-073dfb6844d023bca

Description
-

Created on
2026-04-17 14:26:19.776000

Last modified
2026-04-17 14:26:19.776000

Class name
-

Tags (0)

Manage tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value.
You can use tags to search and filter your resources or track your AWS costs.

Key	Value
No tags	
No tags	

Your jobs (0) Info

Actions | Run job

Filter jobs

Job name	Type	Created by	Last modified	AWS Glue version	Action

Activate Windows
Go to Settings to activate Windows.

Query 1 | Query 2

```sql
CREATE TABLE customer_orders (
    orderid INT,
    userid INT,
    firstname VARCHAR(50),
    lastname VARCHAR(50),
    productname VARCHAR(50),
    quantity INT,
    price INT,
    totalprice INT,
    customerage INT,
    orderdate DATE
);
```

Query 1 | Query 2

```sql
select * from customer_orders;
```

Rows returned (5)

Export

Search rows

1

firstname	lastname	productname	orderdate	orderid_long	price_long	customerage_long	userid_long	totalprice_long	quantity_long
Tom	Holland	Shoes	2024-05-13	114	50	29	7	200	4
Robert	Downey	Watch	2024-05-09	109	120	56	8	480	4
Chris	Evans	Watch	2024-05-12	113	120	37	6	600	5
Emma	Watson	Hat	2024-05-06	106	15	32	5	75	5
John	Doe	Phone	2024-05-15	116	600	30	1	3000	5

The transformed dataset is loaded into Redshift for analytics.

Steps:

- Created table customer_orders

- Used Glue job with JDBC connection

- Data written using temporary S3 directory

Purpose:

- Enables faster analytical queries

- Supports BI tools integration

- Suitable for large-scale data warehousing